CS4433 – Moderated Learning Management System

Phase 6: Quality Assurance Documentation

1. Static Analysis Report

1.1 Tool Used

Tool: Cppcheck
Purpose: Static analysis of the C++ source code to identify potential programming errors, style issues, performance concerns, and portability problems.

Cppcheck was executed against the src and tests directories using the following command:

```
cppcheck --enable=warning,style,performance,portability \
    --check-level=exhaustive \
    --inline-suppr \
    --std=c++17 \
    --language=c++ \
    -I include \
    --suppress=missingIncludeSystem \
    src tests \
    2> cppcheck-exhaustive-report.txt
```

1.2 Analysis Results

The exhaustive analysis produced the following observations:

Finding category	Result
Critical programming errors	None reported
Performance issues	None reported
Portability issues	None reported
Style suggestions	Reported
GoogleTest fixture warnings	3 reported

The analysis did not report critical errors, performance issues, or portability issues. However, it identified several opportunities to improve code readability and maintainability.

1.3 Style Findings

The main style suggestions included:

- Replacing certain loops with standard library algorithms such as std::find_if, std::any_of, std::copy, and std::copy_if.
- Using const references for parameters that do not need to modify the objects passed to them.
- Using standard algorithms such as std::transform, std::all_of, and std::none_of where appropriate.

These suggestions were primarily identified in main.cpp, CreateAccount.cpp, and CourseRepositoryTest.cpp.

They are recommendations rather than confirmed functional defects. They were reviewed with consideration for the existing project architecture and readability.

1.4 GoogleTest Fixture Findings

Cppcheck reported three unusedStructMember findings involving members in the following test fixtures:

- AuthenticationServiceTest
- LatePenaltyPolicyTest
- ThreeDayGraduatedLatePenaltyPolicyTest

These members are used by GoogleTest test cases. The findings are considered false positives because Cppcheck does not fully recognise their use through the GoogleTest fixture mechanism.

The members were therefore retained.

1.5 Conclusion

The exhaustive Cppcheck analysis did not identify critical programming errors, performance issues, or portability issues. The remaining findings consist mainly of optional style improvements and GoogleTest fixture false positives.

The analysis report was saved as:

cppcheck-exhaustive-report.txt

The initial analysis report was also retained as:

cppcheck-report.txt

---

2. Code Metrics Report

2.1 Tool Used

Tool: Lizard 1.25.0

Lizard was used to measure source code size, function count, average cyclomatic complexity, and functions with high cyclomatic complexity.

The following command was used:

lizard src include

The results were saved using:

lizard src include > code-metrics.txt

2.2 Overall Code Metrics

Metric	Result
Files analysed	48
Total non-comment lines of code (NLOC)	4,795
Function count	158
Average function NLOC	26.8
Average cyclomatic complexity (CCN)	5.4
Average token count	131.1
Functions exceeding the CCN threshold	13

The results indicate that the project contains 158 functions across 48 analysed files, with an average cyclomatic complexity of 5.4.

2.3 Key File Metrics

File	NLOC	Average CCN	Functions
src/main.cpp	1,377	8.8	35
src/repositories/SubmissionRepository.cpp	1,116	8.0	27
src/repositories/AssignmentRepository.cpp	219	9.2	5
src/services/CreateAccount.cpp	128	9.0	4

2.4 Functions with High Cyclomatic Complexity

Lizard reported 13 functions exceeding its configured cyclomatic complexity threshold.

Function	Cyclomatic complexity
ExtensionRepository::grantIndividualExtension	19
parseDate	20
SubmissionRepository::gradeSubmission	26
SubmissionRepository::moderateSubmission	16
SubmissionRepository::releaseSubmission	16
SubmissionRepository::changeSubmissionMark	20

CreateAccount::createAccount	26
isValidDate	17
gradeSubmission	19
processHODSubmission	18
changeHODMark	21
manageUsers	26
main	41

2.5 Interpretation and Recommended Improvements

Cyclomatic complexity measures the number of independent paths through a function. A higher value can indicate that a function contains many decisions and may be harder to test and maintain.

The main() function has the highest reported complexity, with a CCN of 41. Other functions with relatively high complexity include SubmissionRepository::gradeSubmission, CreateAccount::createAccount, and manageUsers.

Recommended improvements include:

- Breaking large functions into smaller functions with clearly defined responsibilities.
- Separating menu navigation from business logic.
- Moving repeated validation operations into reusable helper functions.
- Simplifying nested conditional statements where possible.
- Keeping database operations separate from console input and output.

These improvements should be implemented carefully and followed by regression testing to ensure existing functionality is preserved.

2.6 Conclusion

The Lizard results provide a quantitative overview of the project's size and complexity. Although the average cyclomatic complexity is 5.4, several individual functions have higher complexity and should be considered for future refactoring.

The metrics report was saved as:

code-metrics.txt

3. Self-Code-Review Checklist

The following checklist was used to review the implementation before final submission.

Review item	**Status**

Project follows the intended modular architecture	Reviewed
Models, repositories, and services have separate responsibilities	Reviewed
Database operations are handled through repositories	Reviewed
Passwords are hashed using libsodium	Tested
User authentication validates credentials	Tested
Role-based menus are provided	Tested
Student course enrolment is supported	Tested
Assignment submission is supported	Tested
Late submission penalties are applied	Tested
Individual assignment extensions are supported	Tested
Lecturer grading functionality is available	Tested
HOD moderation functionality is available	Tested
Results can be released to students	Tested
Automated tests execute successfully	Passed
Cppcheck static analysis was performed	Completed
Lizard code metrics were generated	Completed
Git branching and CI configuration are available	Completed
High-complexity functions have been identified for improvement	Reviewed

Note: "Reviewed" means the item was examined; it does not necessarily mean every possible improvement has been implemented.

---

4. Code Review Record

4.1 Review Summary

A self-code-review was conducted to evaluate the structure, reliability, maintainability, and quality of the Moderated LMS implementation.

The review considered the project's modular architecture, database interaction, authentication, role-based functionality, assignment submission, grading, moderation, and result release.

Static analysis and code metrics were also used to identify areas requiring attention.

4.2 Review Findings and Actions

Finding	**Action or decision**
Cppcheck suggested replacing some loops with standard algorithms	Reviewed as optional maintainability improvements
Cppcheck suggested const references for some parameters	Reviewed for possible improvement
GoogleTest fixture members were reported as unused	Retained because they are used by test cases
Several functions have high cyclomatic complexity	Recorded for future refactoring

main() has a cyclomatic complexity of 41	Identified as a priority for future decomposition
Submission and moderation operations contain complex decision logic	Identified for possible separation into smaller functions

4.3 Changes Made During Quality Assurance

The following quality assurance activities were completed:

1. Added a GitHub Actions workflow to automate project configuration, compilation, and testing.
2. Configured the CI environment to use MSYS2 UCRT64 and the required C++ dependencies.
3. Ensured that the test target links against the required libsodium dependency.
4. Removed generated build artifacts from Git tracking.
5. Updated .gitignore to exclude build outputs, database files, coverage files, and the Python virtual environment.
6. Ran Cppcheck to inspect the source code.
7. Ran Lizard to measure code size and complexity.
8. Saved the static analysis and code metrics reports for project evidence.

4.4 Outstanding Improvements

The following improvements remain recommended:

- Refactor the most complex functions, especially main().
- Reduce duplicated conditional logic in grading and moderation operations.
- Consider applying appropriate standard library algorithms to improve readability.
- Continue regression testing after any structural changes.
- Repeat static analysis and code metrics after significant refactoring.

4.5 Review Conclusion

The self-code-review confirmed that the project has a modular structure and that its main academic workflows have been implemented and tested.

Automated testing, static analysis, and code metrics were used to assess the implementation. The findings provide a basis for further improvements in maintainability and complexity management.

5. Quality Assurance Evidence

The following files and records provide supporting evidence for the Phase 6 quality assurance activities.

Evidence	File or location
Initial Cppcheck report	cppcheck-report.txt
Exhaustive Cppcheck report	cppcheck-exhaustive-report.txt
Lizard code metrics	code-metrics.txt
Automated test results	CTest output
CI build and test results	GitHub Actions
Source code and configuration	GitHub repository

Repository:
https://github.com/Kevin-droid408/CS4433-Moderated-Learning-Management-System

CI workflow:
https://github.com/Kevin-droid408/CS4433-Moderated-Learning-Management-System/actions

6. Overall Phase 6 Quality Assurance Conclusion

The Moderated LMS underwent configuration management and quality assurance activities to improve the reliability and maintainability of the project.

Git was used for version control, with a branching strategy separating stable releases, integration work, and individual feature development. GitHub Actions was configured to build the application and execute automated tests on pushes and pull requests.

Cppcheck was used to identify potential source code issues, while Lizard provided measurements of source code size and cyclomatic complexity. A self-code-review was also conducted to identify areas for improvement.

The results demonstrate that the project has an automated build and test process and that its code quality has been assessed using static analysis and complexity metrics. Remaining style suggestions and high-complexity functions have been documented for further improvement.